Συστήματα Διαχείρισης Πληροφορίας
Class Projects (2026-27, Fall Semester)

Contents

0. General requirements (all projects) 2

1. Comparison between Vector DataBases 3
   - Vec1: qdrant, milvus 4
   - Vec2: milvus, weaviate
   - Vec3: chroma, pgvector
   - Vec4: pgvector, VectorChord

2. Comparison between text-to-SQL methods 5
   - LLMSQL1: Qwen3, Phi-4-mini, SQLite, PostgreSQL 6
   - LLMSQL2: OmniSQL, Qwen2.5-Coder, SQLite, PostgreSQL
   - LLMSQL3: Gemma 3, Qwen3, PostgreSQL, MySQL

3. Evaluation of Approximate Nearest Neighbor (ANN) Index Algorithms 7
   - ANN1: HNSW, IVF-PQ, DiskANN/Vamana 8
   - ANN2: HNSW, IVF, ScaNN (or USearch) – filtered search
   - ANN3: FAISS quantization family, RaBitQ, binary quantization

4. Evaluation of Graph Embedding Methods on Real-World Datasets 9
   - EMB1: Graph2Vec, NetLSD, GIN over MUTAG - ENZYMES - IMDB-MULTI 10
   - EMB2: NetLSD, FGSD, InfoGraph over REDDIT-MULTI-12K - NCI1 - MUTAG
   - EMB3: Graph2Vec, FGSD, GIN over ENZYMES - IMDB-MULTI - REDDIT-MULTI-12K

5. Data Valuation with Shapley-based Methods: Quality vs. Computational Cost 11
   - DV1: LOO, TMC-Shapley, KNN-Shapley 12
   - DV2: Beta Shapley, Data Banzhaf, Data-OOB
   - DV3: exact Shapley (small n), TMC-Shapley, a stratified or chunk-based estimator

0. General requirements (all projects)

The following apply to every project and every combination. They are part of the grading.

1) **Resources:** Teams are encouraged to use local resources. Every experiment should run on a personal computer from 2020 or later (indicatively: 4–8 cores with AVX2 or ARM NEON, 16 GB RAM, SSD, no GPU required). No cloud resources / paid services are needed. Each combination is sized so that the complete set of experiments takes about **1–2 days of machine time** on such a PC. Where a project asks for data that "do not fit in main memory", create this regime either realistically or by limiting the memory available to the system (e.g., `docker run --memory=4g`). Free notebook GPUs (Colab, Kaggle) may be used where a project says so, but timings obtained on different hardware must not be compared with each other.
2) **Operating system:** Some tools are available only for Linux (noted in each project). On Windows use WSL2 or Docker Desktop. On macOS use Docker.
3) **Hardware and software report:** Report the CPU model, number of cores, SIMD support (AVX2, AVX-512 or NEON), RAM, disk type, operating system, and the exact versions of all systems and libraries. Provide a Dockerfile and/or a `requirements.txt` / environment file with pinned versions.
4) **Reproducibility:** The report must state the repository URL and the **git tag or commit hash** of the submitted code, together with scripts/howtos that run each experiment end to end. Use fixed random seeds and state them.
5) **Statistically valid measurements:** Repeat every timing measurement (at least 5 runs), report the median (or mean) with a 95% confidence interval or the standard deviation, and state what error bars show. Define cold and warm runs and the warm-up you discard. For latency report percentiles (p50, p95, p99), not only averages. State whether your client issues requests one after the other (closed loop) or at a fixed rate (open loop), and with how many concurrent clients. Deterministic quality metrics (e.g., accuracy at temperature 0) need not be repeated.
6) **Fair comparison:** Compare systems at equal quality (e.g., equal recall, equal accuracy, equal durability settings) and with comparable tuning effort. Document every non-default configuration.

1. Comparison between Vector DataBases

In this project, you will be asked to compare the performance of various aspects of two popular and open-source vector database systems. This entails the following aspects that must be tackled by you:

1) **Installation and setup of two specific Vector DBs:** Using your own PC, you are asked to successfully install and setup the two systems in single-node mode (e.g., using Docker). Distributed (cluster) editions are not mandatory for this project (but welcomed if you actually do it).
2) **Data generation (or discovery of real data) and loading to the two DBs:** Using either a specific data generator, online data or artificially creating data, you should identify and load a significant amount of vectors into the databases. Loaded data should: a) have varying dimensionality (vector size and type - integer, float), with at least one dataset of **modern text embeddings** (384–1024 dimensions), b) be the same in both databases, and c) be up to about millions of vectors per dataset. In at least one experiment, limit the memory available to the DB (Section 0) below the size of the data and index, and compare the behavior. Data loading is a process that should be monitored, namely: the time it takes to load a small, medium and the final amount of data, the **index build time**, the storage space it takes in each of the databases (i.e., how efficient data compression or possible indexing is), and the peak memory used.
3) **Query generation to measure performance:** A set of *similarity queries* (common to both DBs) must be compiled in order to test the performance of the vector DBs. Queries should target similar similarity metric algorithms (if implemented on both DBs, e.g., Euclidean distance (L2), Inner product, Cosine similarity, etc.) with and without filters (ann-filtering-benchmark-datasets). Filters should have controlled **selectivity** (e.g., 50%, 10% and 1% of the vectors pass the filter). Use the exact answers (ground truth) provided with the datasets, or compute them with brute-force search, so that recall can be measured.
4) **Measurement of relevant performance metrics for direct comparison:** Client process(es) should pose the queries of the previous step and measure the DB performance: **recall@10**, query latency (p50/p95/p99), throughput (QPS) with 1 and with 8 concurrent clients, CPU and memory load. Since every system can trade accuracy for speed through its index parameters (e.g., HNSW ef), comparisons must be made **at equal recall levels**: produce recall–QPS curves by varying the search parameters (4–6 values are enough), and report e.g. the QPS at recall 0.90 and 0.95, not the numbers obtained with default settings. Teams should be careful and compare meaningful statistics in this important step (see Section 0).

Teams undertaking this project can take advantage of existing benchmarking code either in principle or in whole. I suggest looking at the qdrant Benchmark Suite (vector-db-benchmark), which for many of the DBs contains code for data generation, loading, queries and measurement. Note that it is maintained by one of the vendors: check and report the configuration it uses for each system. Datasets with precomputed ground truth are available from ann-benchmarks (e.g., SIFT1M, GloVe-100) and from VIBE (Vector Index Benchmark for Embeddings), which focuses on modern embedding datasets.

Besides the aforementioned project aspects, you are free to improvise in order to best demonstrate the relative strengths and weaknesses of each system. By the end of your

project, you should have a pretty good idea of what a modern vector DB is, its data and query processing model and convey their strong/weak points.

This project has 4 different DB combinations:

Vec1: qdrant, milvus

Vec2: milvus, weaviate

Vec3: chroma, pgvector

Vec4: pgvector (HNSW, with iterative index scans for filtered queries), VectorChord: vector search inside PostgreSQL

Also possible (all free, open source): LanceDB (Apache-2.0), Vespa (Apache-2.0), OpenSearch k-NN (Apache-2.0), DuckDB with the vss extension (MIT), Redis 8 (AGPL-3.0).

2. Comparison between text-to-SQL methods

With the advent of Large Language models (LLMs), the task of translating natural language into SQL (text-to-SQL) has gained significant attention in the Database community. LLMs such as Qwen, Llama, GPT, etc., have shown superior performance in converting text into SQL queries. However, for complex queries and large schemas the performance tends to drop. As part of this work, you are asked to compare how LLMs perform when generating both simple and complex SQL queries from a text for a Relational Database Management System (RDBMS). Additionally, you are required to develop and scale code to create an agent that integrates LLMs with an RDBMS and compare the performance across different types of queries, SQL dialects and prompting methods. This entails the following aspects that must be tackled by you:

1) **Installation and setup of two specific systems:** Using your own PC you are asked to successfully install and setup the LLM environment and the RDBMS environment. All models of the combinations are free open-weight models and must run locally in quantized form (4-bit, through Ollama or llama.cpp). On a CPU, a 4B model needs roughly 20–90 seconds per question when the full BIRD database schema is in the prompt (less with schema pruning). 7–8B models are about twice as slow, so for them use a GPU if you have one, or a free notebook GPU (Section 0). Measure the speed of your setup on 10 questions first and plan the experiments accordingly.
2) **Data Generation:** Using existing text-to-SQL benchmarks and creating your own set with more complex SQL queries, you should identify the data that will be used for testing the system. Use: a) 150 questions of BIRD Mini-Dev, stratified by its difficulty levels (simple, moderate, challenging). BIRD Mini-Dev provides the same questions for SQLite, MySQL and PostgreSQL, so no translation is needed. b) 100 questions of the Spider 1.0 dev set, run on SQLite only, as a baseline. c) your own set of at least 30 questions over one of the BIRD databases, with queries that include multiple joins, nesting, aggregation and window functions, written for both RDBMSs of your combination.
3) **Code for measuring performance:** Using (python) scripts, you must implement a small framework that acts as an agent for natural language input, generated SQL queries, and execution on the RDBMS. The results produced by the system should then be used to compare the performance of different LLMs in generating queries. Performance must be evaluated in terms of accuracy and computational efficiency (time, tokens and resources used). Accuracy must be measured as **Execution Accuracy (EX)**, i.e., by comparing the result sets of the generated and the gold query, not the SQL strings. Each team is encouraged to go beyond the minimum requirements by designing additional experiments, such as introducing multiple levels of query complexity.
4) **Methods, not only models:** For each model, compare zero-shot prompting with **one** improved method: few-shot prompting, schema pruning (giving the model only the relevant tables and columns), or self-correction with execution feedback (the error message is returned to the model for one more attempt).
5) **Statistically valid results:** Use temperature 0, report bootstrap confidence intervals for EX, and use a paired test (McNemar) when comparing two models or methods on

the same questions. Analyze the errors by category (joins, nesting, aggregation, schema linking, dialect errors).

Teams undertaking this project must use at least two RDBMSs, with different SQL dialects; the dialect differences you meet (e.g., date functions, `LIMIT`, strictness of `GROUP BY`, case sensitivity) are part of the analysis, and SQLGlot can help with your own queries. Similarly, each team should evaluate at least two LLM models. Teams are free to explore additional free models beyond the ones in their combination. Paid API models (e.g., GPT) are not required; if a team uses one, it must be within a free tier and its results must be reported separately. Besides the aforementioned project aspects, you are free to improvise.

Note: Once the work is completed, we encourage teams to back up and save their results, including the generated SQL of every question.

This project has the following different combinations:

LLMSQL1: Qwen3 (4B), Phi-4-mini (3.8B), SQLite, PostgreSQL

LLMSQL2: OmniSQL (7B), Qwen2.5-Coder (7B), SQLite, PostgreSQL: how does a model trained for SQLite transfer to PostgreSQL?

LLMSQL3: Gemma 3 (4B), Qwen3 (4B), PostgreSQL, MySQL

3. Evaluation of Approximate Nearest Neighbor (ANN) Index Algorithms

Approximate nearest neighbor (ANN) search is the core of modern vector databases and retrieval-augmented systems. While Project 1 compares complete vector database *systems* (server, API, ingestion, filtering), this project looks *inside* them: you are asked to evaluate and compare the **index algorithms** themselves (graph-based, partition/quantization-based, disk-based) using stand-alone libraries, and to explain why each behaves as it does. This entails the following aspects that must be tackled by you:

1) **Installation and setup:** Install the libraries of your combination on your own PC (Docker or WSL2 is recommended, since some libraries are available only for Linux).
2) **Data:** Use three datasets from ann-benchmarks or VIBE, of up to about 1M vectors each, that differ in dimensionality and similarity metric (L2, inner product or cosine): e.g., SIFT1M (128-d, L2), GloVe-100 (angular), and one **modern text embedding dataset** (384–1024 dimensions; a 1M subset if larger). Use the ground-truth neighbors that come with the datasets.
3) **Index construction:** Measure build time, index size, peak memory, and the effect of the main construction parameters (e.g., HNSW `M` and `efConstruction`, IVF `nlist`, PQ `m`, Vamana `R` and `L`). 2–3 values per parameter are enough.
4) **Search evaluation:** Report **recall@10 vs. QPS curves** (single-threaded) and p95/p99 latency, and the effect of the search parameters (`efSearch`, `nprobe`, re-ranking depth). Comparisons must be made at **equal recall levels** (e.g., QPS at recall 0.90 and 0.95), not at arbitrary parameter settings. Also report multi-threaded QPS at one recall level.
5) **Beyond Recall–QPS (one of the following, as set by your combination):**
    a) *Memory-constrained regime:* limit the RAM (e.g., with Docker) below the size of the in-memory index and compare in-memory, compressed and disk-based indexes.
    b) *Quantization with re-ranking:* scalar, product and binary quantization, and RaBitQ; recall loss vs. memory savings, with and without re-ranking.
    c) *Filtered search:* attribute filters with controlled selectivity (e.g., 1%, 10%, 50%); compare pre-filtering, post-filtering and filtering inside the index search (FAISS `IDSelector`, hnswlib filter functions).

Existing frameworks (ann-benchmarks, big-ann-benchmarks) may be used for the data and the measurement harness, but the experiments and the analysis must be your own. Results depend strongly on the CPU (SIMD instructions such as AVX2, AVX-512 or NEON), so all compared methods must run on the same machine. By the end of your project, you should be able to explain which index to choose for a given dataset, memory budget and accuracy target.

This project has 3 different combinations:

ANN1: HNSW (hnswlib or FAISS), IVF-PQ (FAISS), DiskANN/Vamana, with the memory-constrained regime (step 5a)

ANN2: HNSW (hnswlib or FAISS), IVF (FAISS), ScaNN (or USearch), with filtered search (step 5c)

ANN3: FAISS (Flat, SQ, PQ, HNSW), RaBitQ (FAISS), binary quantization with re-ranking, with quantization (step 5b)

4. Evaluation of Graph Embedding Methods on Real-World Datasets

Graph embedding methods map entire graphs into fixed-length vector representations that capture structural and semantic information. With such representations we can do tasks like graph classification, clustering, and visualization. Modern whole-graph embedding algorithms include unsupervised approaches (e.g., Graph2Vec, NetLSD, FGSD), self-supervised deep learning models (e.g., InfoGraph) and supervised end-to-end models (e.g., Graph Isomorphism Networks – GIN). This project plans to benchmark several representative embedding techniques on real-world graph datasets, analyzing their relative performance, scalability, and robustness. In more detail, the following aspects must be tackled by you:

1) **Implement or integrate existing open-source implementations** using Python libraries such as KarateClub (for Graph2Vec, NetLSD, FGSD) and PyTorch Geometric (for GIN). KarateClub has not been updated since 2022 and depends on older versions of numpy and networkx: install it in a separate, pinned environment. For InfoGraph, either use an existing implementation in its own pinned environment (the original code or PyGCL, both written for older PyTorch Geometric versions) or re-implement it on the current PyTorch Geometric (a GIN encoder plus a mutual-information objective, about 150 lines). Ensure all methods produce embeddings of comparable dimensionality for fair evaluation. As a baseline, also run the **Weisfeiler–Lehman (WL) subtree kernel** with an SVM (e.g., with GraKeL). On small datasets, it is often competitive with GNNs.
2) **Use standard benchmark graph classification datasets** from the TUDataset collection, as given by your combination. IMDB-MULTI and REDDIT-MULTI-12K have no node features or labels: use node degrees as features, and as node labels for Graph2Vec and WL.
3) **Evaluate performance on three different tasks:**
    a) Classification
        i) Use **stratified 10-fold cross-validation**, with hyperparameters selected only on the training folds from a small grid (at most 4 settings; see Errica et al., *A Fair Comparison of Graph Neural Networks for Graph Classification*, ICLR 2020). Repeat the cross-validation 3 times on the small datasets, which have high variance across folds. For REDDIT-MULTI-12K, a single run with 5 folds and fixed hyperparameters is enough.
        ii) For the unsupervised and self-supervised methods, use embeddings as input features to train a simple classifier (SVM, MLP, etc.) for each dataset. Train GIN end-to-end as the supervised reference.
        iii) Report classification accuracy, macro-F1 and macro one-vs-rest AUC on test data.
        iv) Record training time, generation time and memory use for each method.
        v) Vary embedding dimensions (3 values, e.g., 32, 128, 512) to analyze accuracy vs. compute cost.
    b) Clustering
        i) k-means clustering of the embeddings.

ii) Report ARI and NMI, and qualitative t-SNE/UMAP visualizations.

iii) Which embeddings yield the clearest cluster separation?

c) Stability Analysis (on one small and one large dataset)

i) Introduce random perturbations: add or remove 5% and 20% of the edges.

ii) Recompute embeddings. How much do they change w.r.t. the original? Embeddings from different runs are not aligned (most methods start from a random initialization), so do not compare vectors directly: measure the overlap of each graph's 10 nearest neighbors before and after the perturbation (Jaccard). Use the overlap between two random seeds without any perturbation as the reference.

iii) Report this stability score and the change in classification accuracy.

Graph embedding methods:

- KarateClub whole-graph embedding
- graph2vec, NetLSD, InfoGraph, PyGCL
- PyTorch Geometric GIN, GraKeL (WL kernel)

Datasets:

- TUDataset

Small datasets - MUTAG, ENZYMES, IMDB-MULTI. Large datasets - REDDIT-MULTI-12K, NCI1.

All methods can be trained on a CPU for these datasets. REDDIT-MULTI-12K (about 12,000 graphs with about 390 nodes on average) is the most demanding one: use mini-batches and at most 50 epochs, and report the training time. Colab's free GPU tier may be used in addition (Section 0). Very helpful Colab templates: PyG Colab notebooks.

This project has 3 different combinations (all of them also include the WL kernel baseline):

EMB1: Graph2Vec, NetLSD, GIN over MUTAG - ENZYMES - IMDB-MULTI

EMB2: NetLSD, FGSD, InfoGraph over REDDIT-MULTI-12K - NCI1 - MUTAG

EMB3: Graph2Vec, FGSD, GIN over ENZYMES - IMDB-MULTI - REDDIT-MULTI-12K

5. Data Valuation with Shapley-based Methods: Quality vs. Computational Cost

The quality of training data determines the quality of any model trained on it. *Data valuation* assigns to every training point a value that reflects its contribution to a model's performance; low values point to mislabeled, noisy or redundant data, high values to the points that matter most. The best-known approach, the **Data Shapley** value, comes from cooperative game theory and has desirable properties (fairness axioms), but computing it exactly requires training a model on every subset of the data, i.e., 2^n subsets for n points. Practical methods therefore approximate it, by Monte Carlo sampling of permutations (TMC-Shapley), by exploiting a specific model (KNN-Shapley has a closed form), by changing the weighting of subsets (Beta Shapley, Data Banzhaf), or by avoiding retraining altogether (Data-OOB). In this project, you are asked to benchmark such methods and answer the central question: **how much valuation quality do we get for how much computation?** This entails the following aspects that must be tackled by you:

1) **Installation and setup:** Install OpenDataVal and/or pyDVL, which implement the methods of all combinations. Use logistic regression as the utility model for all methods you compare (it trains in a few milliseconds on these data sizes), and fixed training/validation/test splits.
2) **Data:** Use three datasets: two tabular (e.g., electricity, 2dplanes or MiniBooNE from OpenML, available through OpenDataVal) and one of pre-computed image or text embeddings (e.g., CIFAR10 or IMDB embeddings in OpenDataVal), so that no GPU is needed. Use training sets of 500 and 2,000 points for the sampling-based methods (TMC-Shapley, Beta Shapley, Data Banzhaf). The cheap methods (LOO, KNN-Shapley, Data-OOB) should also be run on 10,000 points. Inject **label noise** at known rates (5%, 10%, 20%), so that the "bad" points are known.
3) **Evaluation of valuation quality:**
    a) *Noisy label detection:* mark the lowest-valued points (as many as the injected noise) and report precision, recall and F1 against the truly corrupted points.
    b) *Point removal:* remove points in decreasing and in increasing order of value, retrain the model, and plot test accuracy against the fraction removed. Good values make accuracy drop fast when high-value points are removed first.
4) **Measurement of computational cost and stability:** For every method, report wall-clock time, number of utility evaluations (model retrainings) and peak memory, and how they grow with n. Run each method with 3 random seeds and report the rank correlation (Spearman) of the values between runs. For the sampling-based methods, on one dataset, plot how the values and the noisy-label F1 converge as the number of sampled permutations or subsets grows.
5) **Quality vs. cost:** Combine steps 3 and 4 into quality–cost plots (e.g., noisy-label F1 against time) and discuss when a cheap method (e.g., KNN-Shapley, Data-OOB) is good enough and when the expensive ones pay off.

As an indication of cost: with logistic regression, TMC-Shapley with 200 permutations on 2,000 points needs a few hundred thousand retrainings, i.e., roughly 10–25 minutes on a 2020 PC per dataset, noise rate and seed. Besides the aforementioned project aspects, you are free to

improvise, e.g., by applying the best method to a real dataset with unknown errors and inspecting the lowest-valued points.

References:

- A. Ghorbani, J. Zou. Data Shapley: Equitable Valuation of Data for Machine Learning. ICML 2019 (TMC-Shapley).
- R. Jia et al. Efficient Task-Specific Data Valuation for Nearest Neighbor Algorithms. PVLDB 12(11), 2019 (KNN-Shapley).
- Y. Kwon, J. Zou. Beta Shapley: a Unified and Noise-reduced Data Valuation Framework for Machine Learning. AISTATS 2022.
- J. T. Wang, R. Jia. Data Banzhaf: A Robust Data Valuation Framework for Machine Learning. AISTATS 2023.
- Y. Kwon, J. Zou. Data-OOB: Out-of-bag Estimate as a Simple and Efficient Data Value. ICML 2023.
- K. F. Jiang et al. OpenDataVal: a Unified Benchmark for Data Valuation. NeurIPS 2023 Datasets and Benchmarks.

This project has 3 different combinations:

DV1: Leave-One-Out (LOO), TMC-Shapley, KNN-Shapley

DV2: Beta Shapley, Data Banzhaf, Data-OOB

DV3: exact Shapley on a training set of 12 points (all 4,096 subsets) compared with TMC-Shapley and a stratified (pyDVL) or chunk-based estimator (values computed over chunks of the data, as in chunked Data Shapley; own implementation), with focus on the convergence and cost analysis of step 4